Mathematical appendix: PH log-survival scaling

Mathematical reference for `_scale_log_survival`. The guards below preserve the PH formula where information is available; real-arithmetic approximation bounds exclude rounding and primitive errors.

Quantities and evaluated shapes

For baseline survival S_0, CDF F_0 and cumulative hazard H_0, write

$$\ell_0 = \log S_0 = -H_0, \qquad a = \exp(\eta), \qquad \ell = a\ell_0 = -aH_0.$$

Here ℓ is PH log survival, and η is the log hazard ratio.

Code quantity	Meaning
`log_s`	Baseline log survival ℓ_0
`baseline_log_cdf`	Optional $\log F_0$, not a probability
`log_baseline_cumulative_hazard`	$\log H_0$ on valid nonzero inputs
`log_scaled_cumulative_hazard`	$\log(aH_0) = \eta + \log H_0$
`cumulative_hazard_magnitude`	Magnitude of the scaled cumulative hazard

Neither of the log cumulative hazard variables denotes log instantaneous hazard. The helper takes evaluated arrays, not time. If the distribution batch shape is B, its stored predictor has shape B. The two array inputs have evaluated shape E: scalar-time calls and the infinity hook use $E = B$; pointwise calls use the broadcast of time shape and B; shared grids use $E = (K, *B)$. Both inputs therefore include the grid axis when present. The infinity hook omits the CDF input. Outputs have shape E; component axes do not occur.

Ordinary finite negative inputs use

$$\ell = -\exp\{\eta + \log(-\ell_0)\}.$$

Adding logarithms before exponentiation avoids premature overflow of a. For float32, $r = 10^{-30}$, $t = 1$ and $\eta = 90$ give $a \approx 1.22 \times 10^{39}$, which overflows alone, while $aH_0 = art \approx 1.22 \times 10^{9}$ is representable. The code retains input sign with `abs`/`sign`; it does not infer parameter validity.

Lower-tail recovery

If H_0 underflows, `log_s` may be zero while a finite baseline log CDF still retains its magnitude. Taylor expansion gives

$$F_0 = 1 - e^{-H_0} = H_0 - \frac{H_0^2}{2} + O(H_0^3), \qquad \log F_0 = \log H_0 - \frac{H_0}{2} + O(H_0^2).$$

The helper then uses $\log F_0$ in place of $\log H_0$, yielding $\tilde{\ell} = -aF_0$ instead of $\ell = -aH_0$. For $H_0 > 0$,

$$\frac{F_0}{H_0} = \int_0^1 e^{-uH_0}\, du, \qquad 0 \le 1 - \frac{F_0}{H_0} \le \int_0^1 uH_0\, du = \frac{H_0}{2}.$$

Thus the relative value error is at most $H_0/2$ in real arithmetic. Scaling by a does not enlarge that relative bound. The same bound holds for the predictor derivative because $\partial_\eta \ell = \ell$ and $\partial_\eta \tilde{\ell} = \tilde{\ell}$. For a baseline parameter θ with nonzero $\partial_\theta H_0$,

$$\partial_\theta \tilde{\ell} = -ae^{-H_0}\partial_\theta H_0, \qquad \frac{|\partial_\theta \tilde{\ell} - \partial_\theta \ell|}{|\partial_\theta \ell|} = 1 - e^{-H_0} \le H_0.$$

These are local real-arithmetic bounds, not uniform floating-point or Hessian guarantees. The approximation is used only when `log_s == 0` and the supplied `baseline_log_cdf` is finite. No epsilon, floor or altered time is introduced.

For $r = 10^{-30}$, $t = 10^{-20}$ and $\eta = 40$ in float32, $rt = 10^{-50}$ underflows, but $art \approx 2.354 \times 10^{-33}$ is normal. An accurate log CDF retains $\log F_0 \approx \log r + \log t$, recovering both this value and $\partial_r \ell = -at \approx -2.354 \times 10^{-3}$. If the generic log CDF was computed from an already rounded-zero log survival, it may instead be $-\infty$: its magnitude is unavailable, so recovery is not selected.

Safe operands and overflow

Every mask is entrywise. `boundary` includes rounded zeros at positive times; it is an implementation label, not exclusively a physical time boundary.

Mask	What it selects
`zero`	Exact or rounded zero baseline log survival
`recover`	Zero with a finite supplied log CDF
`boundary`	Unrecoverable zero or infinity, finite predictor
`overflow`	Scaled log magnitude at or above `log(max)`
`linear`	Boundary zero whose exponential factor is representable

The code substitutes -1 for baseline log survival at special/recovery entries before taking its log magnitude. It substitutes 0 for the predictor on `boundary` entries. These placeholders allow harmless discarded calculations. Recovery entries then replace the placeholder log magnitude by the active log CDF.

For the final exponential, an overflowing input is replaced by zero first:

```
exp(where(overflow, 0.0, log_scaled_cumulative_hazard))
```

The signed infinity is restored afterwards; the result is not clipped to a finite cap. `max` is the calculation dtype's largest finite number, with $\log(\max) \approx 88.72$ for float32 and 709.78 for float64.

A final `where` around an unsafe exponential is insufficient: reverse-mode derivatives can contain $0 \times \infty$ even when its forward value is discarded. Batching can evaluate both branches. Substitution must precede the unsafe logarithm, exponential or multiplication. This protects finite outputs such as $S(0) = 1$ and proper-baseline $S(\infty) = 0$ from inactive overflow tangents; it does not supply useful derivatives at an infinite result.

Zero-limit tangents and promotion

A zero value need not have a zero baseline derivative. If $\ell_0 = 0$,

$$\partial_\theta \ell = a\, \partial_\theta \ell_0.$$

Returning literal zero loses this tangent; returning `log_s` unscaled loses a. When recovery is unavailable and a is representable, the helper uses a safe linear product. Both factors are masked before multiplication:

```
zero_value = where(linear, log_s, 0.0) * exp(where(linear, eta, 0.0))
```

This keeps the baseline tangent even when a generic log CDF cannot recover the value. It cannot reconstruct a missing value or predictor derivative from a zero.

For baseline log cure fraction $c = \log p$, PH survival at infinity is

$$S(\infty) = \exp\{ac\} = p^a.$$

At $c = 0$, the value is 1 while $\partial_c S(\infty) = a$. If $p = 1$ is held fixed, the predictor derivative is zero for every finite η. At $\eta = 1000$, forming $0 \exp(\eta)$ would give NaN. The bound in `linear` avoids this expression. It makes no guarantee for an unrepresentable baseline tangent.

The predictor is promoted with `log_s` before exponentiation. A float64 baseline and float32 $\eta = 90$ therefore retain the representable float64 exp(90) tangent. Stored parameter dtypes and global JAX precision are unchanged.

Callers and information-loss limits

The ordinary `log_survival` caller supplies the real-time log CDF only at zero baseline log survival. Elsewhere its inactive CDF evaluation uses the stop-gradient baseline reference time. The helper does not stop gradients on active model calculations. Base policy supplies the time-zero survival limits and selects the infinity hook separately.

The infinity hook supplies baseline log survival without a CDF argument. A proper baseline's $-\infty$ limit is retained for every finite predictor, even if a separately evaluated a would underflow to zero. Finite negative log cure fractions use ordinary scaling; a zero limit uses the guarded linear route. The special-value mask requires finite η, preventing a NaN predictor from silently acquiring a valid proper limit. Parameter diagnostics remain separate.

If both baseline representations lost the lower-tail magnitude, only the available linear tangent can be preserved. Likewise, an input $-\infty$ might mean true zero survival or prior overflow of a finite baseline cumulative hazard. The helper preserves that infinity; it cannot reconstruct a hidden finite magnitude even if negative η would make the true PH result representable. These guards are not a proof of arbitrary tail Hessians or a repair for inaccurate custom primitives.